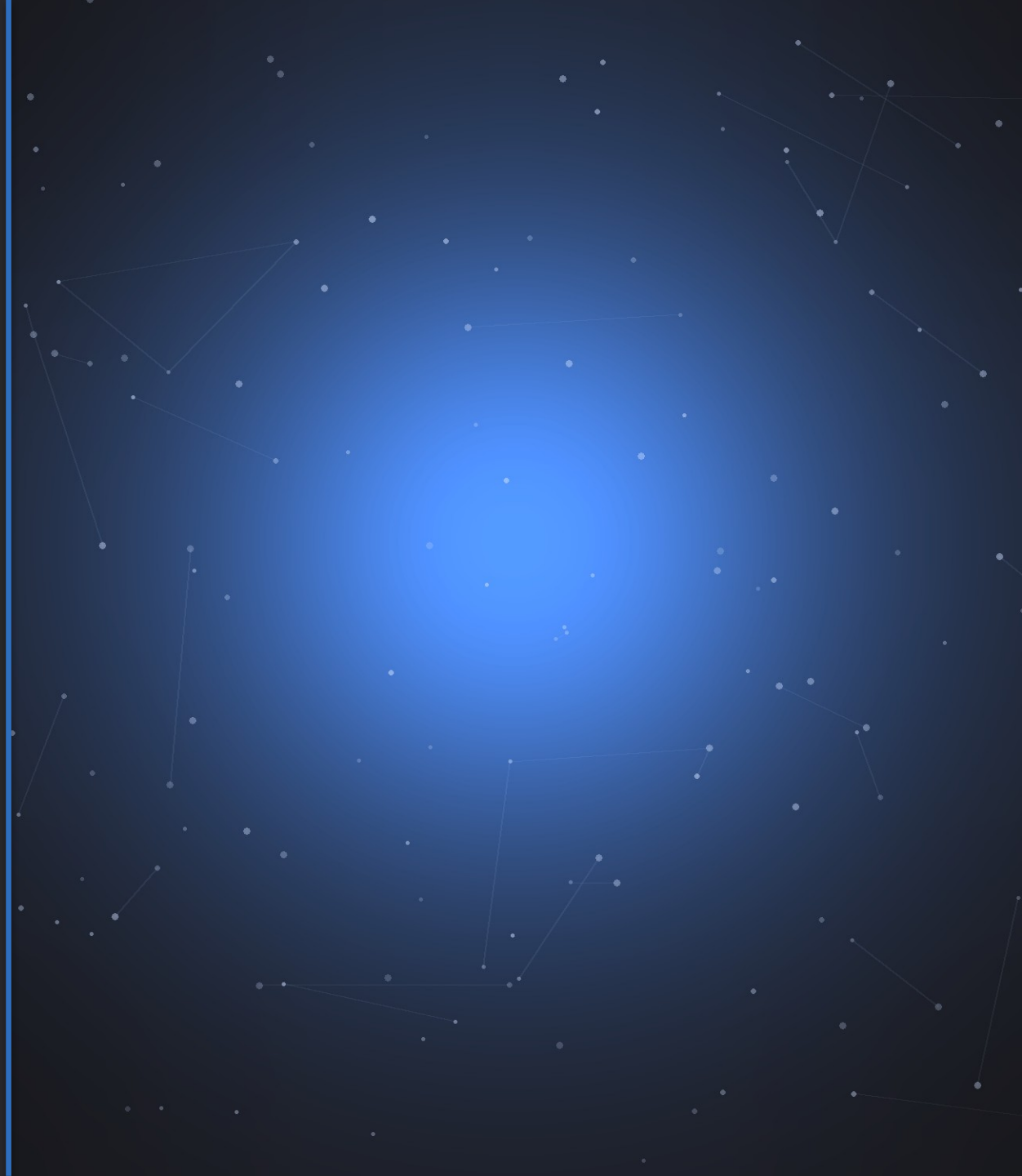

AIA COPILOT TRAINING

Claude Code: Fundamentals

Mastronardi Produce | Aug 11 / 19 /
26, 2026

Scott Hay



Day 1 - five blocks, 9:00 to 4:00

01 The agentic loop & setup

9:00-10:30 - Install, models, extended thinking, sessions, first loop

02 Memory & CLAUDE.md

10:40-12:10 - Teach it once; state lives in files. Lunch 12:10-12:55

03 Context mastery

12:55-2:20 - Aim context, rewind vs revert, recover fast

04 Commands, cost, permissions, git

2:30-3:40 - Auto mode blast radius, ship a reviewed PR

05 Wrap-up

3:40-4:00 - Recap, five mistakes, homework

THE MISSION

What you will build today

One continuous artifact - a business dashboard that Claude Code builds with you across four chained labs.



Lab 1A - scaffold the dashboard from one prompt



Lab 1B - teach Claude the codebase with CLAUDE.md



Lab 1C - break it on purpose, recover with rewind



Lab 1D - verify the feature, commit, pull request

OUTCOMES

By four o'clock you can



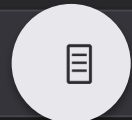
Install & authenticate

Native installer on your OS, signed in, and Claude answering.



Run the loop

Explain the five stages and watch Claude run them on real code.



Write a CLAUDE.md

A lean project brief Claude loads every single session.



Scope context

@-mention exactly what matters; one task per session.



Set guardrails

Cycle the four permission modes and write a deny/ask/allow rule.



Ship a PR

Commit and open a pull request without leaving the chat.



All labs and handouts: aiacopilot.com/claudecode - bookmark it now, you will use it all three days.



BLOCK 1 - 90 MINUTES

The Agentic Loop & Setup

Install it, license it, and understand the engine underneath.

Agentic is table stakes



Every modern tool

- Copilot & Cursor run agent modes too
- Background agents open PRs from issues
- The loop is now the industry model



Why Claude Code here

- Whole loop exposed - CLI, IDE, CI, SDK
- Primitives: memory, skills, hooks, plugins
- Orchestration: subagents to agent teams



These skills transfer: every major tool runs this loop - Claude Code just lets you drive all of it.

Your seat: enterprise, ready



One identity

Sign in with your organization account - the same identity everywhere Claude runs.



Org policy applies

Admins can pin models and permission rules via managed settings - some options may be preset for you.



Classroom gate

Launch Claude Code, run `/model` and `/usage` - green before Lab 1A.



Everyone here is on the enterprise plan - no licensing homework, just sign in and verify.

Install: one command, no Node

PLATFORM	COMMAND
VS Code extension	Extensions panel -> "Claude Code" (Anthropic). Bundles its own CLI.
macOS / Linux / WSL	<code>curl -fsSL https://claude.ai/install.sh bash</code>
Windows PowerShell	<code>irm https://claude.ai/install.ps1 iex</code>
Windows WinGet	<code>winget install Anthropic.ClaudeCode</code>
macOS Homebrew	<code>brew install --cask claude-code</code>

FOLLOW ALONG

First run: prove it works

Do these in order

- 1. Make a folder, open it in VS Code
- 2. Start Claude and sign in
- 3. Confirm your model
- 4. Give it a first instruction

THEN PASTE THESE

1

claude

2

/model

3

Create hello.md with one line explaining what Claude Code is, then tell me what you did.

Where you'll run it



VS Code panel (our setup)

The full Claude Code panel - five modes incl. Auto, plus an Effort slider. Use this one.



Narrow side-pane tab

VS Code's Agent Host runs Claude through an adapter - normalized session model, so some Anthropic controls do not appear.



Integrated terminal

Same engine, text interface. Our fallback, and where Day 3's headless/CI work lives.



Terminal CLI standalone

The canonical surface - new features land here first.



Desktop app / JetBrains

Both exist; JetBrains needs the CLI already on PATH.



Same engine everywhere. The panel and the terminal name a few controls differently - we will flag those as they come up.

One menu, the whole course



Context

Attach file, mention a file, clear the conversation, rewind - Block 3 this afternoon.



Model

Switch model, Effort, Thinking, Account & usage - the next few slides, and cost in Block 4.



Customize: Memory, Permissions

CLAUDE.md and the rules you write - Blocks 2 and 4 today.



Customize: Hooks, Plugins

Tomorrow, Day 2.



Customize: Agents, MCP servers

Day 3.



Items with a terminal icon still work in the panel - they are configured from the CLI.

Output styles: make it explain



Default

Completes tasks efficiently, concise responses. Where you are now.



Proactive

Executes immediately, minimizes interruptions, prefers action over planning.



Explanatory

Explains its implementation choices and your codebase patterns. Use this today.



Learning

Pauses and asks YOU to write code. Aimed at learning to code - not our goal here.



Explanatory while you are learning the agent. Default when you are shipping.

Session anatomy: two ways in



Interactive

- A REPL - read, evaluate, print, loop
- Today's default for real work
- Resume later: `--continue` / `--resume`



One-shot

- `claude -p` - print mode
- One prompt in, result out
- Scripts and CI - Day 3 territory



Start clean when the task changes - resuming imports old context. Verify resume flags at delivery.

BLOCK 1 - MODELS

Verify the default, then choose deliberately

The `default` alias depends on account type, billing mode, provider, and administrator policy.



Sonnet 5 - daily implementation and debugging; common subscription default



Opus 5 - complex reasoning; default for several premium, API, and cloud paths



Your control - use /model to inspect and switch for the task

The model menu, August 2026

ALIAS	USE IT FOR
sonnet / Sonnet 5	Daily implementation and debugging; common subscription default
opus / Opus 5	Complex reasoning and architecture; requires Claude Code v2.1.219+
fable / Fable 5	Hardest and longest autonomous tasks; availability and safety fallback vary
haiku / latest Haiku	Fast, low-cost mechanical work and lightweight scanning

Effort: how much work, not thinking

Anthropic's guidance: use the default. Reach for the dial deliberately.



What it controls

Files read, verification done, steps taken before it checks back in.



Start at default

Set it as a standing preference for your work - not task by task.



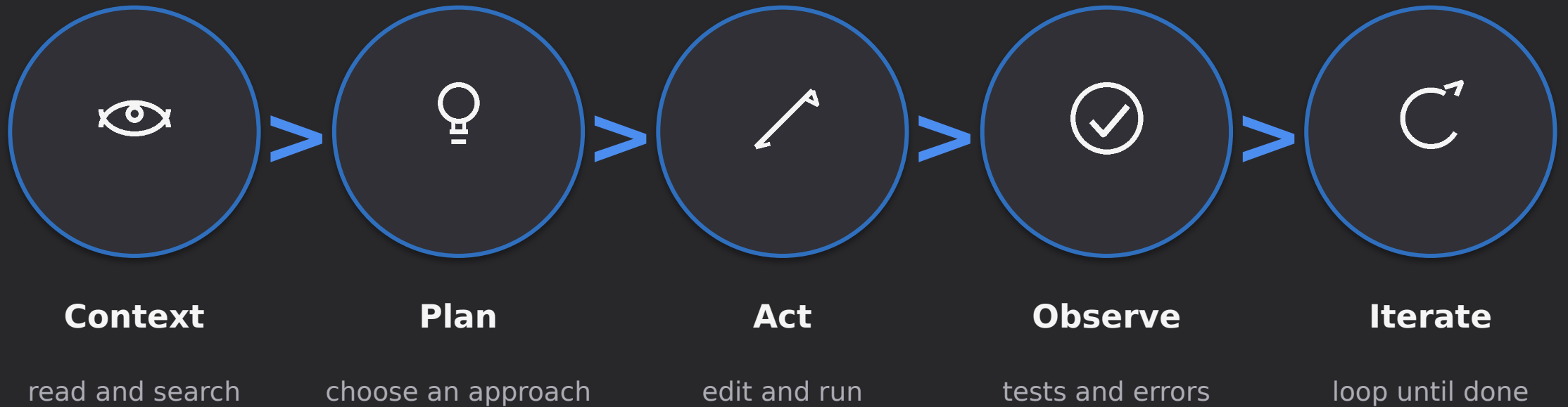
The diagnostic

Didn't try hard enough?
Effort. Didn't know enough?
Bigger model.



Levels run low through max. Effort shapes token use; it does not cap it.

The agentic loop



You press Enter - then what?

01

Read

Files and
memory.

>

02

Plan

An
approach
set.

>

03

Edit

Files
written.

>

04

Run

Commands
run.

>

05

Fix

Errors
retrieved.

When the loop breaks



Symptoms

- Same mistake repeatedly
- Quality drops mid-session
- Earlier instructions forgotten



Fixes

- Smaller task, fresh session
- Rewind past the wrong turn
- Make the rule permanent in CLAUDE.md



The full recovery ladder - Esc, rewind, /compact, /clear - is Block 3's centerpiece.

Principle 1: Small and testable

The habit every other habit builds on.



Small

One task per session - not 'refactor the whole app'.



Frequent

Commit after each change;
/clear between tasks.



Actionable

Every prompt produces a result you can test.



Write this down - Principles 2, 3 and 4 later today all build on this one.

THE SHIFT

**Stop writing prompts.
Start writing loops.**

The tool system



Read / Write / Edit

File tools - surgical changes, not wholesale rewrites.



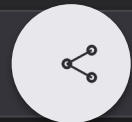
Glob & Grep

Find files by pattern; search content across the repo.



Bash

Any shell command - tests, builds, git, scripts.



WebFetch & WebSearch

Live documentation and answers from the web.



Agent

Spawns subagents for parallel work - Day 3 territory.



+ your own

MCP servers add tools to this list - your APIs, databases, internal systems (Day 3).



Anthropic dropped vector search for plain grep here - the simple tool won. Test simple first.

/doctor: when something feels wrong

Your first move when the tool misbehaves - not a routine you run every day.



Full diagnostic

Install, auth, config, MCP - one health screen.



Auto-fix

Press f and it repairs what it just found.



Flags waste

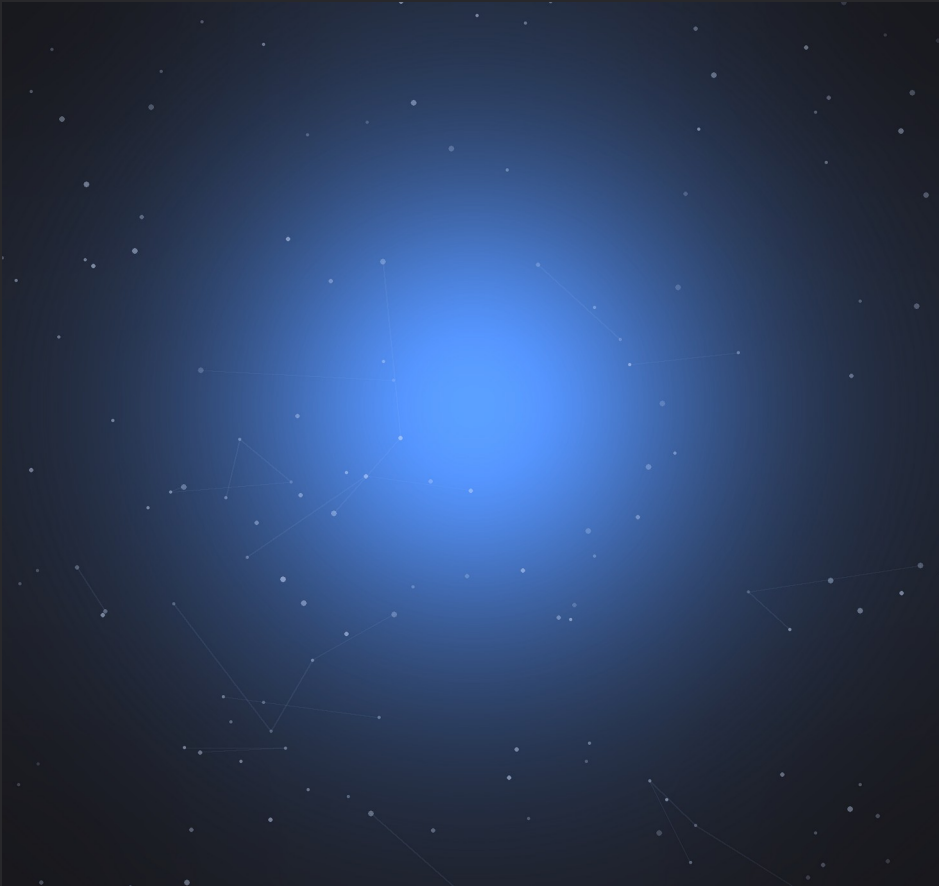
Unused skills, unused MCP servers, slow hooks.



The report is long and machine-specific. Read the section about your problem; ignore the rest.

LIVE DEMO

Demo: the loop on real code



WHAT WE'LL WATCH TOGETHER

- 1** Open the repo first - always
- 2** One prompt, then hands off the keyboard
- 3** Read, plan, edit, run, fix - live

Lab 1A: The Agentic Build Loop

Build the dashboard scaffold and watch the loop work.



Deliverable

A running business-dashboard scaffold built from one prompt.



Timebox

30 minutes, hands-on. If behind: reach a passing first build.



Success checklist

App runs; five loop stages observed; one checkpoint noted.



Debrief

Share one loop failure you watched Claude self-correct.

10 MINUTES

Break



BLOCK 2 - 90 MINUTES

Memory & CLAUDE.md

Teach it once. It remembers every session.

BLOCK 2 - CONCEPT

What Claude can actually see

One rule: if it is not in the context window, Claude does not know it.



Working memory - the context window - the only thing it can see



Session storage - files on your disk - replayed into context on resume



Persistent instructions - CLAUDE.md - re-read at the start of every session

What is CLAUDE.md?

One file, read first, every single session.



A config file

Lives in the project root; read first, every session.



An onboarding doc

Your project's instruction manual for the AI.



Persistent

No more re-explaining the project every chat.



Shareable

Commit it - the whole team gets the same behavior.

What goes in the file



Project & stack

What this is and what it runs on - two lines.



Conventions

Exact rules: 'camelCase variables, PascalCase classes'.



Testing

Framework, coverage bar, where tests live.



Git habits

Commit format, branch naming, PR expectations.



Architecture

Where new code goes; what never gets touched.



Keep it under ~200 lines - Claude follows a lean file far better than a bloated one.

Principle 3: Route, don't pile

Two ways to route: by topic, and by folder.



The pile

One 500-line CLAUDE.md -
half of it gets ignored.



Route by topic

.claude/rules/*.md - testing
rules, API conventions,
security.



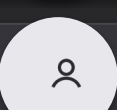
Route by folder

A CLAUDE.md inside a
subfolder - loads only when
Claude reads there.



Subfolder files cost nothing until Claude goes there. Free scoping for a monorepo.

The memory hierarchy

	Enterprise policy	Managed, IT-deployed, org-wide - always wins; cannot be overridden.
	Project CLAUDE.md	Committed to the repo - every collaborator gets it.
	Project rules	.claude/rules/*.md - modular, routed.
	User memory	~/.claude/CLAUDE.md - you, on every project.
	Local override	CLAUDE.local.md - you, this project, gitignored.
	Auto memory	Claude writes this one itself.

 Precedence when rules collide: enterprise > project > user > local - verify exact order at delivery.

Auto memory: Claude's notebook

Claude takes its own notes while it works.



What it saves

Build commands, debugging insights, your preferences.



Where it lives

~/.claude/projects/
<project>/memory -
MEMORY.md index.



How it loads

First ~200 lines read
alongside CLAUDE.md at
start.



Plain markdown - read, edit or delete anytime; can be disabled per project (config file comes in Block 4).

Surviving compaction

Compact Instructions

- A priority list inside CLAUDE.md
- Tells /compact what must survive
- Architecture decisions first

HANDOFF.md

- Claude writes the handoff file
- Tried, worked, failed, next
- Fresh session starts from it



Without guidance, /compact optimizes for brevity - early architecture decisions die first.

BLOCK 2 - THE RULE UNDER IT ALL

State lives in files, not sessions

Rules the pros actually use

From the CLAUDE.md of Claude Code's own creator.



Plan first

'Enter plan mode for ANY non-trivial task.'



Prove it works

'Never mark done without a passing verification.'



Self-improve

'After any correction, write the lesson to lessons.md.'

Four lines that fix the agent



Don't assume

Don't hide confusion. Surface tradeoffs. (fixes: assumes and runs)



Minimum code

Solve the problem. Nothing speculative. (fixes: overbuilds)



Touch only what you must

Clean up only your own mess. (fixes: 40-line diff for a 3-line fix)



Define success criteria

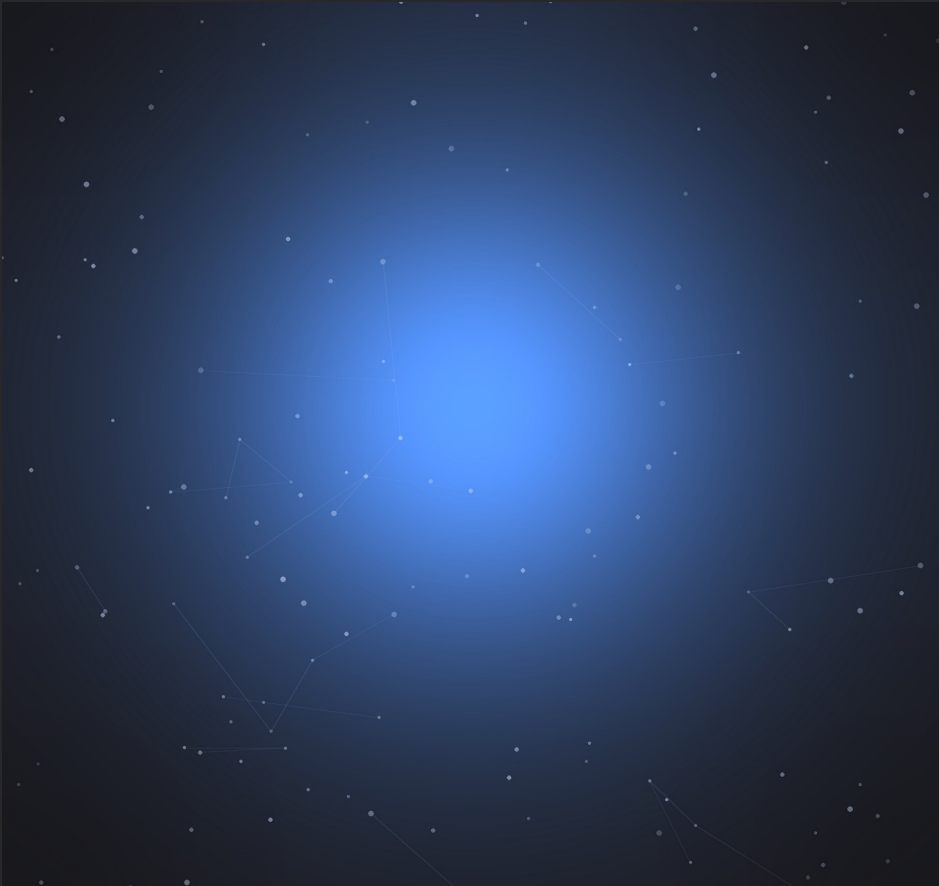
Loop until verified. (fixes: says 'done' without proof)



All four are in the CLAUDE.md Templates handout: aiacopilot.com/claudecode

LIVE DEMO

Demo: /init and auto memory



WHAT WE'LL RUN TOGETHER

- 1 /init on my status-service repo
- 2 Trim the generated file live
- 3 Teach one rule, restart, watch it stick

YOUR TURN

Your turn: teach it something

The two moves I just showed you

- Correct it once - it remembers
- End long work with a handoff file
- Both write to disk, not to a session

RUN THESE IN YOUR DASHBOARD

1

Remember: in this project we use double quotes and 2-space indents. Save that so you never suggest otherwise.

2

Write HANDOFF.md: what you built today, what works, what is unfinished, and what the next session should do first.

Lab 1B: Teach Claude your repo

Turn Lab 1A's scaffold into a taught codebase.



Deliverable

A committed CLAUDE.md the whole team could onboard from.



Timebox

30 minutes. If behind: /init plus three trimmed rules.



Success checklist

Under 200 lines; one taught rule proven to stick after restart.



Debrief

Which rule surprised you by changing Claude's behavior?

RECAP

Two lines to remember

If Block 2 compresses to two sentences, these are them.

“

CLAUDE.md is onboarding documentation that actually gets read - every session.

“

Route, don't pile: two hundred lean lines beat five hundred ignored ones.

45 MINUTES

Lunch



BLOCK 3 - 85 MINUTES

Context Mastery

Feed exactly what matters - and rewind when it goes wrong.

@-mentions: aim, don't spray



@file.ts

Load exactly one file - the default move.



@src/folder/

Loads everything inside - use sparingly.



@https://docs...

Pulls live documentation into context.



Tab completion

Stack multiple mentions in one precise prompt.



One @folder on a big directory can eat your context in a single keystroke.

Aim it, or let it search?

APPROACH	THE TRADE
@-mention the file	Cheapest and most precise - but you must know where the code lives
Let Claude search	Glob and Grep burn turns and tokens - and find what you did not know
Hybrid (the default)	@-mention the entry point, let it search outward from there

Focused or broad?



Focused (default)

- Specific files, one task
- Less context, better output
- Faster and cheaper



Broad (deliberate)

- Architecture and discovery work
- Whole-module refactors
- Expect to /compact sooner

Where your window goes

The context window is a budget - here is the ledger.



Fixed overhead

System prompt, skill metadata, server instructions, and any tools loaded up front.



Semi-fixed

CLAUDE.md and memory - a few thousand tokens.



The rest

Conversation and files - the part you actually control.



MCP tool search defers tool schemas by default; /context shows what actually loaded.

Signs the context is full



Shorter answers

Detail drops; reasoning visibly degrades.



Forgetting

Earlier instructions silently fall away.



Conflicting edits

Changes start fighting each other.



The response

/compact to squeeze - or commit and /clear.

One task per session. Period.

The session-hygiene rule that outranks every other tip.



Context degrades

Long sessions are meetings that never end.



Correction spiral

Two failed fixes = poisoned context. Stop steering.



The rhythm

Commit, /clear, next task.
Every time.



After two failed corrections, rewind or restart - arguing with the session never wins.

Case study: payment module

01

Start

One clear task.



02

Pivot

A second task.



03

Tangle

Both entangle.



04

Burn

Three hours lost.



05

Lesson

Two clean sessions.

BLOCK 3 - NEW IN 2026

Checkpoints: the rewind button

The recovery centerpiece of July 2026 - mistakes are now cheap.



Hover a message - click the rewind arrow
(terminal: double-Esc)



Choose scope - fork conversation, rewind code, or both



Auto checkpoints - created as Claude works - no setup

Rewind: what comes back - what not



Conversation

Wind the chat back to any earlier turn.



Claude's file edits

Code changes restore to the checkpoint.



Shell side effects

Installed packages, migrations, deleted data stay done.



Remote state

Pushed commits and external API calls do not un-happen.



Checkpoints cover the session and Claude's edits - verify exact restore scope at delivery.

The recovery ladder



Esc

interrupt and redirect



Rewind

double-Esc, pick
scope



Compact

squeeze, keep going



Clear

hard reset

QUICK REFERENCE

Which lever, when

SITUATION	REACH FOR
Heading the wrong way	Esc - stop and redirect
Last turns poisoned it	Double-Esc - rewind convo, code, or both
Long session, on track	/compact - summarize and continue
Switching tasks	Commit, then /clear - fresh start

Rewind, or git revert?



Rewind

- Claude's own recent missteps
- Nothing committed or shared yet
- Restores session and code together



git revert / reset

- Committed or pushed history
- Team-visible, auditable undo
- Damage older than your checkpoints



Interrupting: Esc early when direction is wrong; mid-edit, let it finish - half-applied changes cost more.

Principle 2: Next actions

Vague goals in, disappointment out.



Don't

'Fix everything wrong with this codebase.'



Do

'Fix the N+1 query in getUserPosts.'



Verify

Claude is confident even when wrong - check it.



'Almost right' is worse than clearly wrong - it passes review and breaks at 2 AM.

YOUR TURN

Your turn: sharp delegation

Three habits that make a prompt sharp

- Name the file with @ - no hunting
- Fence the scope - 'only that route'
- Demand evidence - run it, show the diff

RUN ONE OF THESE

1

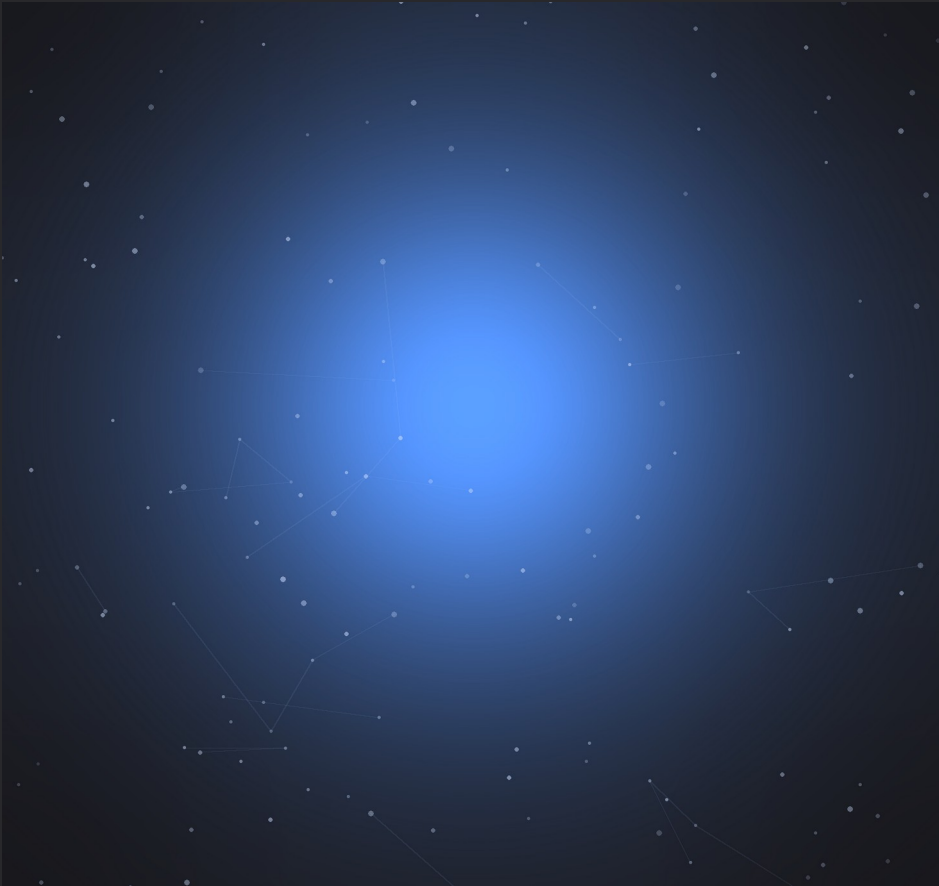
In @server.js, add input validation to the POST /api/metrics route, following the validation already on POST /api/tasks. Then run the server and prove it rejects a bad request.

2

In @server.js, make GET /api/tasks return newest first. Change only that one query, and show me the diff.

LIVE DEMO

Demo: bloat, break, rewind



WHAT WE'LL RUN TOGETHER

- 1** Overload a session on purpose
- 2** Watch quality visibly degrade
- 3** Double-Esc - restore code and continue

Lab 1C: Precision Recovery

Break the dashboard on purpose - then get it back.



Deliverable

A dashboard feature shipped after a deliberate bad detour.



Timebox

30 minutes. If behind: complete one full rewind cycle.



Success checklist

Bad change rewound; rewind-vs-revert call made; /context checked twice.



Debrief

Which scope did you restore - conversation, code, or both? Why?

RECAP

Context in two lines

The whole block, compressed.

“

Less context, better output - feed files, not folders.

“

Rewind beats arguing: double-Esc costs less than three corrections.

10 MINUTES

Break



BLOCK 4 - 70 MINUTES

Commands, Cost & Permissions

Power tools, price tags, and the safety model - then ship it.

Daily-driver commands



/init

Generate the starter CLAUDE.md.



/clear

Hard reset between tasks.



/compact

Compress a long session, keep the thread.



/context

See exactly where your tokens went.



/model

Switch models mid-session - escalate deliberately.



/doctor

Break-glass only - when something is actually broken, not a daily habit.

Quality of life, July 2026

Small features, daily payoff.



/btw

Side question answered without polluting the main context.



! prefix

Run a shell command - Claude reads the output, no second prompt.



/keybindings

Remap everything -
~/.claude/keybindings.json.



These ship in fast-moving weekly releases - re-verify exact behavior on delivery morning.

Build a human-invoked skill

What you are practicing

- Skills are the current custom-workflow form
- `disable-model-invocation: true` keeps it explicit
- `$ARGUMENTS` carries your input
- Commit it for the team

PASTE THIS

1

Create `.claude/skills/review/SKILL.md` with name, trigger description, `disable-model-invocation: true`, and `Read/Grep/Glob` only. Body: review `$ARGUMENTS` and return prioritized findings with file references.

2

`/review server.js`

Spec quality = output quality

⊗ Vague

- 'Build me an API'
- Claude guesses everything
- Three rounds, still wrong

✓ Clear

- Stack, endpoints, auth named
- Matches your mental model
- One round of refinement

The Interview Me pattern

01

Invite

'Interview me
first.'

>

02

Answer

It asks what you
missed.

>

03

Spec

It writes spec.md.

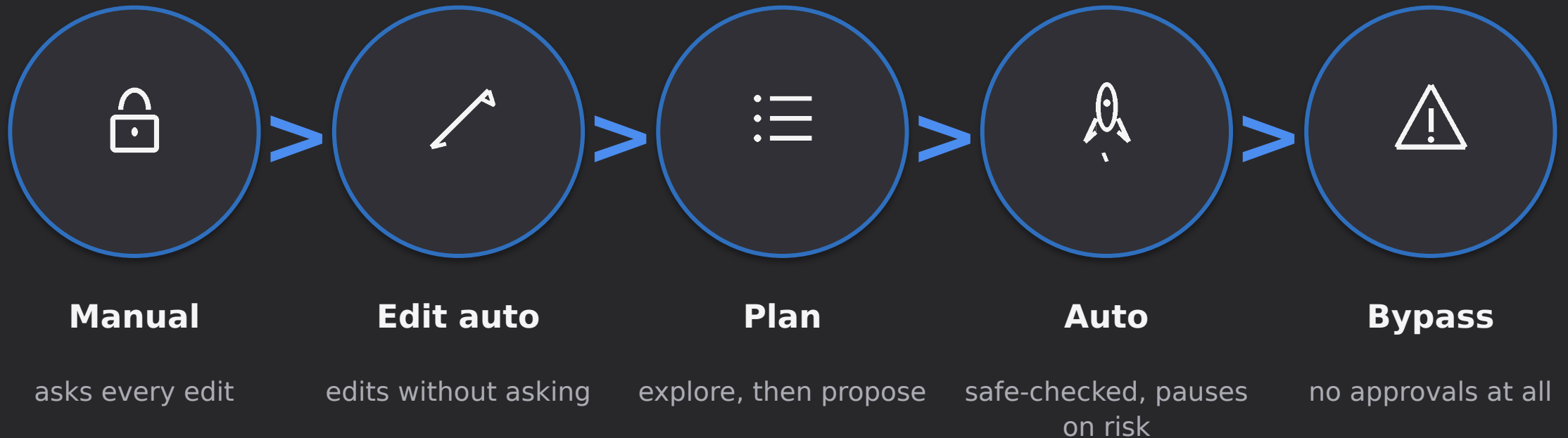
>

04

Execute

Build from the
spec.

Shift+Tab: cycle the modes



Auto mode: guarded autonomy

Two protection layers plus hard guards.



Layer 1: inputs

An injection probe screens what enters the session.



Layer 2: outputs

A transcript classifier reviews what Claude did.



Hard guards

Asks before `rm -rf` on unresolved variables; blocks transcript tampering.



Auto mode is powerful, not unattended - supervise it, and verify guard details at delivery.

Auto mode: shrink the blast radius



Contain the workspace

Sandbox, container, or throwaway clone - not your only checkout.



Starve it of secrets

No production credentials inside an Auto session.



Write the deny list first

With nobody watching, deny rules and guards are the only 'no'.



Size the damage

Assume every allowed action runs - grant accordingly.



Sandboxing options vary by platform and policy - verify available isolation at delivery.

settings.json: where config lives



Enterprise

Managed by your admins. Locked - you cannot override it.



User

~/.claude/settings.json - your defaults, in every project.



Project

.claude/settings.json - committed to the repo, shared by the team.



Local

.claude/settings.local.json - this repo only, gitignored.



Same precedence as the memory hierarchy: more specific wins - except enterprise, which locks.

Rules: deny, ask, allow



Deny - checked first

A matching deny always wins. Bash(rm -rf *), Read(.env).



Ask - the default

Anything unmatched prompts you for a decision.



Allow - pre-approved

Bash(npm test), Bash(git *) - trusted and frictionless.



Where they live

settings.json arrays; managed settings lock org policy.



Evaluation order is deny, then ask, then allow - a matching deny beats an explicit allow.

Principle 4: Default to safe

The posture that keeps permissions enabled.



Asks first

When uncertain, Claude Code prompts. Safety over speed.



Prompts are a feature

Not friction - your review point.



Trust is earned

Start restrictive; loosen rule by rule.

Know which meter you're on

USAGE PATH	CONTROL
Subscription seat	Rolling and weekly allowance shared across Claude surfaces; inspect /usage
API or cloud provider	Pay-per-token billing; use provider budgets and telemetry
Headless / CI	Set --max-turns and --allowedTools; monitor each automation path
Agent teams	Each teammate has its own context, so usage scales with active teammates

Five habits that burn budget



Spraying context

Pasting whole files instead of @-mentioning the two that matter - every token you send is spend.



The immortal session

One conversation all day: repeated compaction, degraded answers, retries - end sessions deliberately.



Premium model, routine work

Running the top-tier model for renames and boilerplate a cheaper model does fine.



Re-explaining the project

Telling Claude your conventions every session instead of writing them once in CLAUDE.md.



No verification step

Skipping the check, shipping a wrong answer, and paying for the whole loop twice.



Every habit on this slide is fixed by something you learned today - the course pays for itself here.

Git to PR, all from chat



See changes

'What have I modified?' - status and diff in plain English.



Commit

Staged with a generated, descriptive message.



Branch

Create, switch, merge - conflicts included.



Pull request

'Create a PR' - description written from your commits.



CI

Reads failing checks and proposes the fix.

Paste this: ship with receipts

What you are practicing

- Agent-assisted review before commit
- Commit message written from the diff
- PR description written from the commits

PASTE THIS

1

Review the staged diff like a skeptical senior engineer: flag risky changes, missed edge cases, and anything unrelated to this task.

2

Commit with a message that says what changed and why, following our commit format.

3

Open a PR. Build the description from the commits: summary, key changes, and how it was tested.

No verifier, no agent



Three levels

Exit codes and tests; integration checks; production signals.



The test

Can you say how Claude knows it is done? If not, do not automate it.

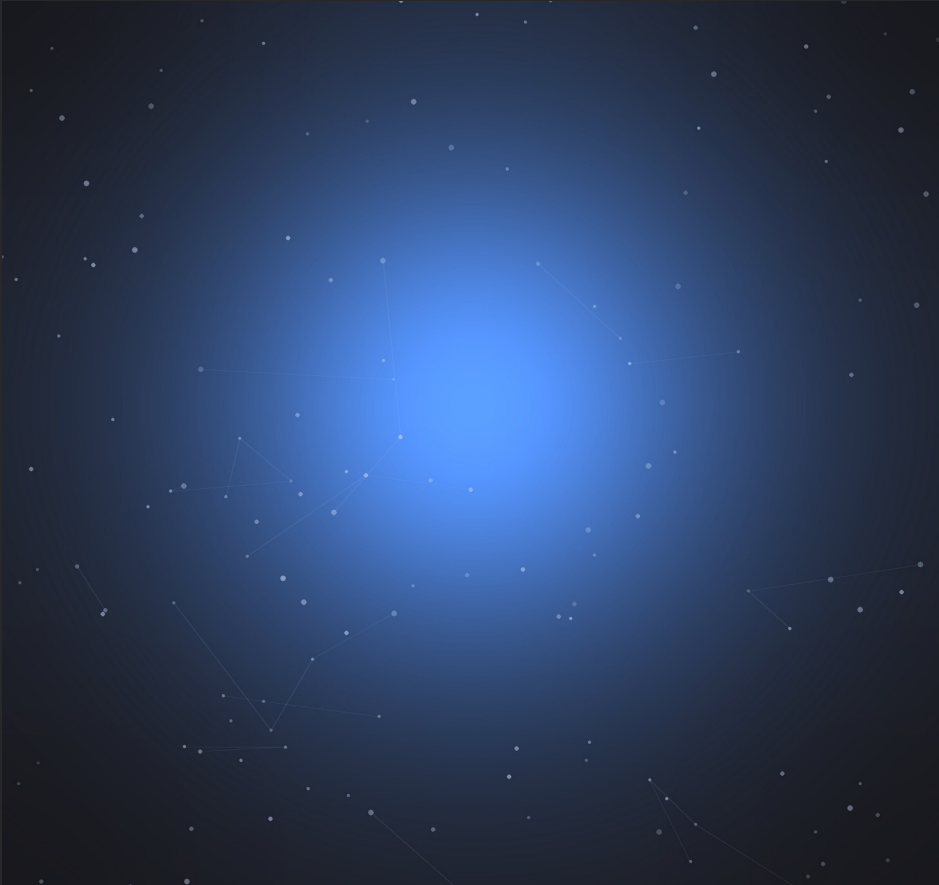


Do this

Define acceptance criteria before the task, not after.

LIVE DEMO

Demo: Auto mode, guarded



WHAT WE'LL RUN TOGETHER

- 1** Shift+Tab to Auto on a safe task
- 2** Trip a guardrail - watch it stop and ask
- 3** A deny rule blocks even an allowed command

HANDS-ON

Lab 1D: Ship It

The dashboard leaves your machine tonight.



Deliverable

Export endpoints verified, committed, PR open for review.



Timebox

25 minutes. If behind: a reviewed diff and a clean agent-written commit.



Success checklist

Permission rule set; diff reviewed; agent-written commit and PR.



Debrief

What did the diff review catch that you would have missed?



BLOCK 5 - 20 MINUTES

Wrap-Up

What you learned, what breaks, and what happens tonight.

GROUP DISCUSSION

Where would you let it run?

Pre-announced before the break. Cameras and mics on - this one is a conversation, not a poll.

“

Which part of our codebase would you let an agent change unsupervised tomorrow?

“

Where would you absolutely not - and what rule would enforce that?

“

What would have to be true before you moved that line?

What you learned today



Setup

Native install, approved identity, live default verified with /model.



The loop

Five stages Claude runs on its own.



Memory

CLAUDE.md plus auto memory - taught once, kept forever.



Context

@-mentions, one task per session, rewind recovery.



Guardrails

Four modes, deny-ask-allow, priced sessions.



Shipping

Git to PR without leaving the conversation.

Five mistakes, five fixes

MISTAKE	FIX
Context overflow	/clear between tasks; feed files, not folders
Correction spirals	Two fails - rewind or restart, never argue
Forgotten rules	Said it twice? Put it in CLAUDE.md
Scope creep	One task per session - commit, /clear, next
Vibe coding	Spec first; plan mode for anything complex

Your three-day journey

01

Day 1

One agent, driven well.



02

Day 2

Your standards, taught.



03

Day 3

Many agents, in production.

Homework (optional)

Twenty minutes tonight locks in the day.



Real CLAUDE.md

Run /init on your actual project; trim to the essentials.



One explicit skill

Create a human-invoked /workflow you will really use.



One rewind

Practice a deliberate double-Esc recovery on real work.

CLOSING THOUGHT

Leave with these

Day 1 in two sentences.

“

Small, frequent, actionable - the loop rewards discipline.

“

Rewind beats arguing. Specs beat vibes.
Verify beats trust.

WRAP-UP

See you tomorrow

Tonight

Homework: /init your real project, build one human-invoked skill, practice one rewind. Apply one habit to real work and bring questions.

Tomorrow

Day 2 - teach Claude your standards: skills, hooks, invocation control, and governed plugins.